



## Веб-сайт бара «Пьяный Гейгер»

---

Версия документа: 2.0

Дата составления: февраль 2026 г.

Репозиторий: [github.com/q1est/mdk2](https://github.com/q1est/mdk2) | Ветка: version1.0

Хостинг Frontend: GitHub Pages | Backend: Docker / Hugging Face Spaces

База данных: PostgreSQL 16 (Neon / pgxpool)

Документ составлен на основе исходного кода проекта.

Включает: проектную, разработческую, эксплуатационную и юридическую документацию.

# СОДЕРЖАНИЕ

---

## 1. ПРОЕКТНАЯ ДОКУМЕНТАЦИЯ

- 1.1 Техническое задание
- 1.2 Архитектура системы
- 1.3 Структура сайта и страниц
- 1.4 Диаграммы компонентов

## 2. ДОКУМЕНТАЦИЯ РАЗРАБОТЧИКА

- 2.1 Технический стек
- 2.2 Структура файлов проекта
- 2.3 Frontend: модули и компоненты
- 2.4 Backend: архитектура и пакеты
- 2.5 REST API — эндпоинты
- 2.6 Модели данных (Go structs)
- 2.7 Схема базы данных
- 2.8 Инструкция по развёртыванию

## 3. ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ

- 3.1 Руководство администратора
- 3.2 Руководство пользователя
- 3.3 Устранение неполадок (FAQ)
- 3.4 Журнал изменений (Release Notes)

## 4. ЮРИДИЧЕСКАЯ ДОКУМЕНТАЦИЯ

- 4.1 Политика конфиденциальности
- 4.2 Пользовательское соглашение
- 4.3 Акт приёма-передачи работ

# 1. ПРОЕКТНАЯ ДОКУМЕНТАЦИЯ

## 1.1 Техническое задание

### Общие сведения

Проект — корпоративный имиджевый сайт бара «Пьяный Гейгер» в тематике постапокалипсиса (вселенная S.T.A.L.K.E.R.). Реализован как двухстраничный сайт с Go-бэкендом для обработки бронирований и PostgreSQL-базой данных.

| Параметр            | Значение                                               |
|---------------------|--------------------------------------------------------|
| Название            | Сайт бара «Пьяный Гейгер»                              |
| Репозиторий         | github.com/q1est/mdk2                                  |
| Ветка               | version1.0                                             |
| Frontend URL        | q1est.github.io/mdk2/                                  |
| Backend хостинг     | Hugging Face Spaces (Docker), порт 7860                |
| База данных         | PostgreSQL 16 (Neon, подключение через pgxpool/pgx v5) |
| Go-модуль           | restaurant (go 1.24.0)                                 |
| Версия документации | 2.0 (февраль 2026)                                     |

### Цели и задачи

- Предоставить информацию о заведении онлайн (меню, контакты, режим работы).
- Обеспечить онлайн-бронирование столиков с сохранением данных в PostgreSQL.
- Реализовать корзину и форму оформления доставки на странице меню.
- Привлечь целевую аудиторию — фанатов игровых вселенных (S.T.A.L.K.E.R.).

### Функциональные требования

| ID    | Требование                                                                  | Приоритет | Статус |
|-------|-----------------------------------------------------------------------------|-----------|--------|
| FR-01 | Главная страница: hero, сет дня, СТА, контакты, отзывы                      | Высокий   | ✓      |
| FR-02 | Страница меню с карточками по категориям (закуски/основные/десерты/напитки) | Высокий   | ✓      |
| FR-03 | Корзина с подсчётом суммы и формой оформления заказа                        | Высокий   | ✓      |

| ID    | Требование                                                        | Приоритет | Статус |
|-------|-------------------------------------------------------------------|-----------|--------|
| FR-04 | Модальная форма бронирования столика (6 полей + зона + пожелания) | Высокий   | ✓      |
| FR-05 | POST /api/reservations — сохранение брони в PostgreSQL            | Высокий   | ✓      |
| FR-06 | GET /api/reservations — получение списка броней                   | Средний   | ✓      |
| FR-07 | CORS-заголовки для работы фронта с бэком                          | Высокий   | ✓      |
| FR-08 | Блок «Сет дня» с промокодом                                       | Средний   | ✓      |
| FR-09 | Раздел контактов: адрес, телефон, режим работы                    | Высокий   | ✓      |
| FR-10 | Блок отзывов (статические данные)                                 | Низкий    | ✓      |
| FR-11 | Адаптивная вёрстка (mobile 320px+)                                | Высокий   | ✓      |
| FR-12 | Автоматическое создание таблицы reservations при старте           | Высокий   | ✓      |

## Нефункциональные требования

| ID     | Требование       | Значение                                           |
|--------|------------------|----------------------------------------------------|
| NFR-01 | Порт бэкэнда     | 7860 (Hugging Face) / 8080 (docker-compose local)  |
| NFR-02 | БД               | PostgreSQL 16, SSL (ca-certificates в Docker)      |
| NFR-03 | Go-сборка        | CGO_ENABLED=0, GOOS=linux, статический бинарь      |
| NFR-04 | Docker-образ     | Multi-stage: golang:1.24-alpine → alpine:latest    |
| NFR-05 | Кодировка        | UTF-8                                              |
| NFR-06 | Браузеры         | Chrome, Firefox, Safari, Edge (последние 2 версии) |
| NFR-07 | Мобильная версия | от 320px                                           |

## 1.2 Архитектура системы

Система построена по принципу разделения фронтенда и бэкенда. Frontend — статические HTML/CSS/JS файлы на GitHub Pages. Backend — Go HTTP-сервер в Docker-контейнере, работающий с PostgreSQL через библиотеку pgx v5 (connection pool).

### Компоненты и их взаимодействие

| Компонент           | Технология               | Расположение                  | Роль                                    |
|---------------------|--------------------------|-------------------------------|-----------------------------------------|
| Frontend            | HTML5 + CSS3 + JS (ES6+) | GitHub Pages                  | UI, валидация форм, fetch-запросы к API |
| Backend             | Go 1.24, net/http        | Docker / Hugging Face         | REST API, бизнес-логика, запись в БД    |
| База данных         | PostgreSQL 16            | Neon (cloud) / docker-compose | Хранение броней (таблица reservations)  |
| Connection pool     | pgx/v5 pgxpool           | Встроен в Go-сервер           | Пул соединений с PostgreSQL             |
| Контейнер           | Docker (multi-stage)     | Hugging Face Spaces           | Упаковка и деплой Go-бинаря             |
| Оркестрация (local) | docker-compose           | Локально                      | Подъём app + db для разработки          |

### Поток данных: бронирование столика

- 1. Пользователь нажимает «Бронирование» → открывается модальное окно (bookingModal).
- 2. Заполняет форму: имя, телефон, email, дата, время, кол-во гостей, зона, пожелания.
- 3. script.js валидирует обязательные поля на клиенте.
- 4. fetch POST на /api/reservations с JSON-телом {name, phone, date, time, guests}.
- 5. Go-обработчик ReservationsHandler декодирует JSON в models.Reservation.
- 6. db.Pool.Exec() выполняет INSERT INTO reservations — данные сохраняются в PostgreSQL.
- 7. Бэкенд возвращает 201 Created + {"status":"ok"}.
- 8. Frontend показывает пользователю подтверждение.

## 1.3 Структура сайта и страниц

| URL / Файл     | Название | Секции / содержимое                                                                                                                                            |
|----------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| / (index.html) | Главная  | Hero (.hero), Навигация (.hero-nav), Сет дня (.daily-set), Модал бронирования (#bookingModal), СТА (#booking), Контакты (#contacts), Отзывы (.reviews), Footer |
| /menu.html     | Меню     | Кнопка «На главную», Корзина (#cartModal), Навигация по категориям (nav), Секции: #snacks, #main, #desserts, #drinks, Footer, Модал заказа (#orderModal)       |

**Блюда и напитки (menu.html) — реальные данные**

| Категория | Название                  | Описание                                          | Цена    |
|-----------|---------------------------|---------------------------------------------------|---------|
| Закуски   | Тушёнка «Радость туриста» | Консервированное мясо неизвестного происхождения  | 299 ₽   |
| Основные  | Жаркое из мутантов        | Медленно томится до полной потери агрессии        | 699 ₽   |
| Основные  | Чесночный суп             | Отпугивает мутантов и людей в радиусе трёх метров | 1 099 ₽ |
| Десерты   | Аномальный пирог          | Мутировавшие ягоды. Возможны эффекты              | 499 ₽   |
| Напитки   | Радиационный коктейль     | Светится в темноте. Иногда                        | 249 ₽   |
| Напитки   | Мутационный чай           | Свойства не задокументированы. Возможно похмелье  | 99 ₽    |

# 1.4 Диаграммы компонентов

**Frontend — структура модулей (front\_drawio)**

| Слой                     | Модуль              | Компоненты                                           |
|--------------------------|---------------------|------------------------------------------------------|
| page                     | Страницы            | main, menu, information, table reservation, contacts |
| business logic → booking | Логика бронирования | Booking Feature, Menu Feature                        |
| business logic → menu    | Логика меню         | menu list                                            |
| shared → ui components   | UI-компоненты       | button, input, modal                                 |

| Слой            | Модуль     | Компоненты                        |
|-----------------|------------|-----------------------------------|
| shared → layout | Лейаут     | header, footer, navigation        |
| shared → utils  | Утилиты    | вспомогательные функции           |
| api             | API-клиент | api client, api booking, api menu |
| assets          | Статика    | images, styles                    |

## Backend — Clean Architecture (back\_drawio)

| Слой                           | Пакет / модуль        | Содержимое                                   |
|--------------------------------|-----------------------|----------------------------------------------|
| use case                       | create booking        | Сценарий создания бронирования               |
| use case                       | get menu              | Сценарий получения меню                      |
| business logic<br>→ entities   | menu item,<br>booking | Go-структуры доменных объектов               |
| business logic<br>→ services   | booking service       | Бизнес-логика обработки броней               |
| business logic<br>→ interfaces | interfaces            | Интерфейсы репозитория (абстракции)          |
| storage → db                   | menu, booking         | Реализация работы с PostgreSQL (pgxpool)     |
| storage →<br>migrations        | migrations            | SQL-миграции схемы БД                        |
| api                            | menu controller       | Обработчик HTTP-запросов к /api/menu         |
| api                            | booking controller    | Обработчик HTTP-запросов к /api/reservations |
| api                            | contacts controller   | Обработчик HTTP-запросов к /api/contacts     |
| log                            | —                     | Логирование (log.Println, log.Fatal)         |

## 2. ДОКУМЕНТАЦИЯ РАЗРАБОТЧИКА

### 2.1 Технический стек

| Категория  | Технология          | Версия      | Примечание                                     |
|------------|---------------------|-------------|------------------------------------------------|
| Frontend   | HTML5               | 5           | Разметка, семантические теги                   |
| Frontend   | CSS3                | 3           | Кастомные свойства, Flexbox, Grid, анимации    |
| Frontend   | JavaScript          | ES6+        | fetch API, DOM, модальные окна, корзина        |
| Backend    | Go (Golang)         | 1.24.0      | net/http, encoding/json, os, log, context      |
| Backend    | pgx/v5              | 5.8.0       | PostgreSQL-драйвер + pgxpool (connection pool) |
| Backend    | pgpassfile          | 1.0.0       | Indirect dependency pgx                        |
| Backend    | pgservicefile       | 2024-06-06  | Indirect dependency pgx                        |
| Backend    | puddle/v2           | 2.2.2       | Пул соединений (indirect)                      |
| Backend    | golang.org/x/sync   | 0.17.0      | Синхронизация (indirect)                       |
| Backend    | golang.org/x/text   | 0.29.0      | Текстовые утилиты (indirect)                   |
| БД         | PostgreSQL          | 16          | Основная СУБД                                  |
| DevOps     | Docker              | multi-stage | golang:1.24-alpine → alpine:latest             |
| DevOps     | docker-compose      | —           | Локальная разработка: app + db                 |
| Хостинг FE | GitHub Pages        | —           | Деплой статики, CDN                            |
| Хостинг BE | Hugging Face Spaces | —           | Docker-контейнер, порт 7860                    |
| БД (cloud) | Neon                | —           | Serverless PostgreSQL, SSL                     |

### 2.2 Структура файлов проекта

| Путь       | Тип  | Назначение               |
|------------|------|--------------------------|
| index.html | HTML | Главная страница сайта   |
| menu.html  | HTML | Страница меню с корзиной |



| Путь                    | Тип         | Назначение                                        |
|-------------------------|-------------|---------------------------------------------------|
| styles.css              | CSS         | Единая таблица стилей для обеих страниц           |
| script.js               | JavaScript  | Логика форм, корзины, модалов, fetch-запросов     |
| *.png (6 файлов)        | Изображения | Фотографии блюд и напитков меню                   |
| back/main.go            | Go          | Точка входа: инициализация БД + HTTP-сервер       |
| back/handle/handle.go   | Go          | HTTP-обработчик ReservationsHandler (GET/POST)    |
| back/models/models.go   | Go          | Структура Reservation (доменная модель)           |
| back/db/postgres.go     | Go          | Подключение к PostgreSQL (pgxpool), createTable() |
| back/go.mod             | Go module   | Объявление модуля restaurant, зависимости         |
| back/go.sum             | Go          | Хэши зависимостей                                 |
| back/Dockerfile         | Docker      | Multi-stage сборка Go-бинаря                      |
| back/docker-compose.yml | Docker      | Локальное окружение: app + PostgreSQL 16          |
| architecture/           | Диаграммы   | Архитектурные схемы draw.io (front/back)          |
| .gitattributes          | Git         | Настройки атрибутов репозитория                   |
| README.md               | Markdown    | Краткое описание проекта                          |

## 2.3 Frontend: модули и компоненты

### index.html — ключевые элементы DOM

| Селектор / ID     | Тип            | Описание                                            |
|-------------------|----------------|-----------------------------------------------------|
| .hero             | section/header | Hero-секция: название бара и слоган                 |
| .hero-nav         | section        | Навигационные кнопки (меню, бронирование, контакты) |
| #scrollToBooking  | button         | Открывает модал #bookingModal                       |
| #scrollToContacts | button         | Прокрутка к секции #contacts                        |

| Селектор / ID           | Тип               | Описание                                                                    |
|-------------------------|-------------------|-----------------------------------------------------------------------------|
| .daily-set              | section           | Блок «Сет дня» с промокодом                                                 |
| #bookingModal           | div.booking-modal | Модальное окно формы бронирования                                           |
| #bookingForm            | form              | Форма бронирования (name, phone, email, date, time, guests, zone, comments) |
| #closeBooking           | button            | Закрытие модала бронирования                                                |
| #booking / .booking-cta | section           | СТА-блок «Забронировать столик»                                             |
| #ctaBookingBtn          | button            | Открывает модал бронирования из СТА                                         |
| #contacts               | section           | Адрес: Форт №1 «Штайн», тел: +7 911 475-49-69, часы работы                  |
| .reviews                | section           | Блок из 4 отзывов посетителей                                               |
| footer                  | footer            | Копирайт © Бар «Пьяный Гейгер»                                              |

#### menu.html — ключевые элементы DOM

| Селектор / ID    | Тип     | Описание                                                    |
|------------------|---------|-------------------------------------------------------------|
| .back-home-btn   | a       | Ссылка «← На главную» (href=index.html)                     |
| #cartBtn         | button  | Кнопка корзины с счётчиком #cartCount                       |
| #cartModal       | div     | Модал корзины: список, итого, кнопки «Заказать» и «Закрыть» |
| nav              | nav     | Табы фильтрации: Закуски / Основные / Десерты / Напитки     |
| section.category | section | Секции блюд по категориям (active = видима)                 |
| .add-to-cart     | button  | Добавление позиции в корзину (+)                            |
| #orderBtn        | button  | Открывает форму оформления заказа #orderModal               |
| #orderModal      | div     | Модал заказа: имя, телефон, адрес доставки                  |

## 2.4 Backend: архитектура и пакеты

### Пакет main (main.go)

Точка входа приложения. Инициализирует соединение с PostgreSQL, регистрирует HTTP-маршруты и запускает сервер на порту из переменной окружения PORT (по умолчанию 7860 — стандарт Hugging Face Spaces).

```
db.ConnectPostgres() // подключение + создание таблицы
defer db.Pool.Close()
http.HandleFunc("/api/reservations", handle.ReservationsHandler)
port = os.Getenv("PORT") || "7860"
http.ListenAndServe(":"+port, nil)
```

### Пакет db (postgres.go)

Управляет подключением к PostgreSQL через pgxpool (пул соединений). При запуске читает DATABASE\_URL из переменной окружения, создаёт пул, пингует БД и вызывает createTable() для авто-создания таблицы reservations.

| Функция / переменная | Сигнатура                             | Описание                                      |
|----------------------|---------------------------------------|-----------------------------------------------|
| Pool                 | var Pool *pgxpool.Pool                | Глобальный пул соединений с PostgreSQL        |
| ConnectPostgres()    | func ConnectPostgres()                | Инициализация пула + ping + createTable       |
| createTable()        | func createTable(ctx context.Context) | CREATE TABLE IF NOT EXISTS reservations (...) |

### Пакет handle (handle.go)

Единственный HTTP-обработчик ReservationsHandler. Обработывает три метода: OPTIONS (preflight CORS), POST (создание брони), GET (список броней). Установлены CORS-заголовки для работы с GitHub Pages frontend.

| Метод   | Действие                                                                       | Статус ответа                 |
|---------|--------------------------------------------------------------------------------|-------------------------------|
| OPTIONS | CORS preflight — возвращает разрешённые заголовки и методы                     | 200 OK                        |
| POST    | Декодирует JSON → INSERT INTO reservations (name,phone,date,time,guests)       | 201 Created + {"status":"ok"} |
| GET     | SELECT name,phone,date::TEXT,time::TEXT,guests FROM reservations → JSON-массив | 200 OK + []Reservation        |
| Другой  | Метод не поддерживается (неявно — нет ответа)                                  | —                             |

❗ Поля email, zone, comments из формы фронтенда НЕ передаются в текущий POST-запрос и не сохраняются в БД. В таблице хранятся только: name, phone, date, time, guests.

## 2.5 REST API — эндпоинты

Базовый URL бэкенда: определяется переменной окружения в script.js.

### POST /api/reservations — создать бронирование

| Параметр     | Значение                       |
|--------------|--------------------------------|
| URL          | POST /api/reservations         |
| Content-Type | application/json               |
| CORS         | Access-Control-Allow-Origin: * |

#### Тело запроса (JSON):

```
{ "name": "string", // Имя/позывной — обязательно
  "phone": "string", // Телефон (10 цифр без +7) — обязательно
  "date": "string", // Дата в формате YYYY-MM-DD — обязательно
  "time": "string", // Время в формате HH:MM — обязательно
  "guests": int // Количество гостей — обязательно }
```

#### Ответы:

| HTTP-код                  | Условие                    | Тело ответа        |
|---------------------------|----------------------------|--------------------|
| 201 Created               | Успешная запись в БД       | { "status": "ok" } |
| 400 Bad Request           | Ошибка декодирования JSON  | текст ошибки       |
| 500 Internal Server Error | Ошибка INSERT в PostgreSQL | текст ошибки       |

### GET /api/reservations — получить все бронирования

| Параметр           | Значение                       |
|--------------------|--------------------------------|
| URL                | GET /api/reservations          |
| CORS               | Access-Control-Allow-Origin: * |
| Ответ Content-Type | application/json               |

#### Тело ответа (JSON-массив):

```
[ { "name": "string", "phone": "string",
  "date": "string", "time": "string", "guests": int }, ... ]
```

| HTTP-код                  | Условие                     | Тело ответа                      |
|---------------------------|-----------------------------|----------------------------------|
| 200 OK                    | Успешная выборка            | JSON-массив объектов Reservation |
| 500 Internal Server Error | Ошибка SELECT из PostgreSQL | текст ошибки                     |

## OPTIONS /api/reservations — CORS preflight

Возвращает заголовки Access-Control-Allow-Origin: \*, Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Methods: GET, POST, OPTIONS. Статус 200 OK.

## 2.6 Модели данных (Go structs)

### models.Reservation (models/models.go)

Единственная доменная модель проекта. Используется для декодирования JSON-запросов и сканирования строк из БД.

```
package models

type Reservation struct {
    Name string `json:"name"`
    Phone string `json:"phone"`
    Date string `json:"date"`
    Time string `json:"time"`
    Guests int `json:"guests"`
}
```

| Поле Go | JSON-тег | Тип Go | Тип в БД      | Описание                  |
|---------|----------|--------|---------------|---------------------------|
| Name    | "name"   | string | TEXT NOT NULL | Имя/позывной гостя        |
| Phone   | "phone"  | string | TEXT NOT NULL | Телефон (10 цифр)         |
| Date    | "date"   | string | DATE NOT NULL | Дата визита<br>YYYY-MM-DD |
| Time    | "time"   | string | TIME NOT NULL | Время визита HH:MM        |
| Guests  | "guests" | int    | INT NOT NULL  | Количество гостей         |

## 2.7 Схема базы данных

База данных PostgreSQL 16. Схема создаётся автоматически при старте Go-сервера функцией createTable() в пакете db. Название БД: restaurant.

### Таблица: reservations

Точный SQL из postgres.go:

```
CREATE TABLE IF NOT EXISTS reservations (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    phone TEXT NOT NULL,
    date DATE NOT NULL,
    time TIME NOT NULL,
    guests INT NOT NULL,
```

```
created_at TIMESTAMP DEFAULT NOW()
);
```

| Колонка    | Тип PostgreSQL | Ограничения   | Описание                         |
|------------|----------------|---------------|----------------------------------|
| id         | SERIAL         | PRIMARY KEY   | Автоинкрементный первичный ключ  |
| name       | TEXT           | NOT NULL      | Имя или позывной гостя           |
| phone      | TEXT           | NOT NULL      | Номер телефона (10 цифр, без +7) |
| date       | DATE           | NOT NULL      | Дата визита                      |
| time       | TIME           | NOT NULL      | Время визита                     |
| guests     | INT            | NOT NULL      | Количество гостей                |
| created_at | TIMESTAMP      | DEFAULT NOW() | Метка времени создания записи    |

**i** Поля *email*, *zone* (предпочитаемая зона) и *comments* (особые пожелания) присутствуют в HTML-форме, но не передаются в POST-запрос и не хранятся в БД. При необходимости их можно добавить в модель *Reservation* и в SQL-запрос *INSERT*.

## 2.8 Инструкция по развёртыванию

### А. Локальная разработка (docker-compose)

| Шаг | Команда                                                    | Результат                                                |
|-----|------------------------------------------------------------|----------------------------------------------------------|
| 1   | git clone https://github.com/q1est/mdk2.git && cd mdk2     | Клонирование репозитория                                 |
| 2   | git checkout version1.0                                    | Переключение на рабочую ветку                            |
| 3   | cd back/                                                   | Переход в директорию бэкенда                             |
| 4   | docker-compose up --build                                  | Сборка образа + запуск app (8080) + db PostgreSQL (5432) |
| 5   | # Фронт открывать из корня репозитория напрямую в браузере | index.html → file:// или live-server                     |
| 6   | curl http://localhost:8080/api/reservations                | Проверка доступности API (GET)                           |

**i** `docker-compose.yml` задаёт

`DATABASE_URL=postgres://postgres:password@db:5432/restaurant?sslmode=disable`. Таблица `reservations` создаётся автоматически при первом запуске.

## В. Продакшн-деплой (Hugging Face Spaces + Neon)

| Шаг | Действие                                  | Детали                                                                   |
|-----|-------------------------------------------|--------------------------------------------------------------------------|
| 1   | Создать БД в Neon                         | neon.tech → новый проект → скопировать <code>DATABASE_URL</code> (с SSL) |
| 2   | Создать Space на Hugging Face             | huggingface.co → New Space → Docker SDK                                  |
| 3   | Добавить секрет <code>DATABASE_URL</code> | Space Settings → Repository secrets → <code>DATABASE_URL</code>          |
| 4   | Запустить код бэкенда                     | git push → HF автоматически строит Docker-образ                          |
| 5   | Проверить деплой                          | Space URL + <code>/api/reservations</code> должен вернуть []             |
| 6   | Обновить API URL в <code>script.js</code> | Прописать URL Hugging Face Space в fetch-запросе                         |
| 7   | Задеплоить фронт                          | git push origin version1.0 → GitHub Pages обновится                      |

## С. Переменные окружения

| Переменная                | Обязательность | Значение по умолчанию | Описание                                                                                          |
|---------------------------|----------------|-----------------------|---------------------------------------------------------------------------------------------------|
| <code>DATABASE_URL</code> | Обязательно    | —                     | Строка подключения PostgreSQL. Формат: <code>postgres://user:pass@host:port/db?sslmode=...</code> |
| <code>PORT</code>         | Опционально    | 7860                  | Порт HTTP-сервера. 7860 = стандарт Hugging Face Spaces                                            |

## Д. Dockerfile (multi-stage build)

```
# Этап 1: Сборка
FROM golang:1.24-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./ && RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o main .

# Этап 2: Финальный образ
```

```
FROM alpine:latest
RUN apk --no-cache add ca-certificates # SSL для Neon
WORKDIR /root/
COPY --from=builder /app/main .
ENV PORT=7860
EXPOSE 7860
CMD ["/main"]
```



# 3. ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ

## 3.1 Руководство администратора

### Обновление контента сайта

| Задача                | Файл        | Что редактировать                                                                |
|-----------------------|-------------|----------------------------------------------------------------------------------|
| Изменить сет дня      | index.html  | Блок .daily-set-card → текст .daily-set-description и промокод                   |
| Добавить блюдо в меню | menu.html   | Добавить .item в нужную section.category (#snacks / #main / #desserts / #drinks) |
| Изменить цену блюда   | menu.html   | Найти блюдо по h3, обновить span.price                                           |
| Заменить фото блюда   | Корень репо | Заменить PNG-файл с тем же именем, закоммитить                                   |
| Обновить контакты     | index.html  | Секция #contacts: карточки .contact-card                                         |
| Изменить режим работы | index.html  | Карточка «Режим работы» в .contacts-grid                                         |
| Добавить отзыв        | index.html  | Добавить .review-card в .reviews-grid                                            |

### Деплой обновлений Frontend

- 1. Внести изменения в файлы локально.
- 2. git add . && git commit -m "описание изменений"
- 3. git push origin version1.0
- 4. GitHub Pages обновится автоматически через 1–3 минуты.
- 5. Проверить: <https://q1est.github.io/mdk2/>

### Работа с базой данных

| Операция        | SQL / команда                                            | Примечание                              |
|-----------------|----------------------------------------------------------|-----------------------------------------|
| Просмотр броней | SELECT * FROM reservations<br>ORDER BY created_at DESC;  | Через psql или GUI (TablePlus, DBeaver) |
| Удаление брони  | DELETE FROM reservations<br>WHERE id = ;                 | Осторожно — необратимо                  |
| Кол-во броней   | SELECT COUNT(*) FROM reservations;                       | Быстрая проверка                        |
| Брони на дату   | SELECT * FROM reservations<br>WHERE date = '2026-03-01'; | Фильтр по дате                          |

| Операция            | SQL / команда                                    | Примечание                                   |
|---------------------|--------------------------------------------------|----------------------------------------------|
| Пересоздать таблицу | DROP TABLE reservations; (затем рестарт сервера) | Таблица создастся заново через createTable() |

## 3.2 Руководство пользователя

### Навигация по сайту

| Кнопка / ссылка               | Действие                                            |
|-------------------------------|-----------------------------------------------------|
| «Наше меню»                   | Переход на menu.html — страницу с карточками блюд   |
| «Бронирование»                | Открывает модальное окно формы бронирования столика |
| «Контакты и отзывы»           | Прокрутка страницы к секции контактов               |
| «Забронировать столик» (СТА)  | Также открывает модал бронирования                  |
| «← На главную» (на menu.html) | Возврат на главную страницу                         |

### Бронирование столика

| Шаг | Поле                | Обязательность | Формат / варианты                                         |
|-----|---------------------|----------------|-----------------------------------------------------------|
| 1   | Позывной / Имя      | Обязательно    | Любой текст                                               |
| 2   | Телефон             | Обязательно    | 10 цифр без +7 (например: 9001234567)                     |
| 3   | E-mail              | Необязательно  | Любой e-mail                                              |
| 4   | Дата визита         | Обязательно    | Выбор из календаря                                        |
| 5   | Время               | Обязательно    | Выбор из пикера времени                                   |
| 6   | Количество гостей   | Обязательно    | 1-5 сталкеров / 6+ (группировка)                          |
| 7   | Предпочитаемая зона | Необязательно  | Безопасная зона / У барной стойки / У окна / Укрытый угол |
| 8   | Особые пожелания    | Необязательно  | Свободный текст                                           |

| Шаг | Поле                   | Обязательность | Формат / варианты                       |
|-----|------------------------|----------------|-----------------------------------------|
| 9   | Нажать «Забронировать» | —              | Форма отправляется на /api/reservations |

### Использование меню и корзины

- Перейдите на страницу меню («Наше меню»).
- Используйте вкладки навигации для фильтрации: Закуски / Основные / Десерты / Напитки.
- Нажмите «+» на карточке блюда — позиция добавится в корзину.
- Иконка корзины  отображает счётчик добавленных позиций.
- Нажмите на иконку корзины → откроется список с итоговой суммой.
- Нажмите «Заказать» → откроется форма с полями: имя, телефон, адрес доставки.

## 3.3 Устранение неполадок (FAQ)

### Q: Форма бронирования не отправляется

A: Убедитесь, что все поля со \* заполнены. Телефон: ровно 10 цифр без +7. Откройте DevTools (F12) → вкладка Network — проверьте запрос к /api/reservations. Если статус 5xx — бэкенд недоступен, позвоните: +7 911 475-49-69.

### Q: Ошибка CORS в консоли браузера

A: Бэкенд недоступен или URL в script.js неверный. Проверьте, что сервер запущен и переменная API\_URL указывает на правильный хост.

### Q: Сайт открывается, но фото блюд не загружаются

A: Проверьте имена PNG-файлов в корне репозитория. Имена содержат кириллицу — убедитесь, что файлы закоммичены корректно (git status).

### Q: База данных не подключается при старте

A: Проверьте переменную окружения DATABASE\_URL. Для Neon: строка должна содержать sslmode=require. Для docker-compose: проверьте, что сервис db запустился раньше app (depends\_on).

### Q: Корзина обнуляется при перезагрузке страницы

A: Это ожидаемое поведение — корзина хранится в памяти JavaScript (не в localStorage). При переходе между страницами или перезагрузке данные сбрасываются.

### Q: Docker-контейнер не стартует

A: Проверьте: `docker logs restaurant_app`. Частая причина: `DATABASE_URL` не задан → сервер завершится с `log.Fatal`. Убедитесь, что переменная передана через `environment` в `docker-compose.yml`.

## 3.4 Журнал изменений (Release Notes)

| Версия | Дата      | Тип     | Описание                                                      |
|--------|-----------|---------|---------------------------------------------------------------|
| 1.0.0  | Янв. 2026 | Релиз   | Первый публичный запуск                                       |
| 1.0.0  | Янв. 2026 | Feature | Главная страница: hero, сет дня, СТА, контакты, отзывы        |
| 1.0.0  | Янв. 2026 | Feature | Страница меню с 6 позициями в 4 категориях                    |
| 1.0.0  | Янв. 2026 | Feature | Корзина с подсчётом суммы и формой заказа                     |
| 1.0.0  | Янв. 2026 | Feature | Модальная форма бронирования (8 полей)                        |
| 1.0.0  | Янв. 2026 | Feature | Go REST API: POST/GET <code>/api/reservations</code>          |
| 1.0.0  | Янв. 2026 | Feature | PostgreSQL: таблица <code>reservations</code> , auto-create   |
| 1.0.0  | Янв. 2026 | Feature | CORS-поддержка для работы GitHub Pages + бэкэнд               |
| 1.0.0  | Янв. 2026 | Feature | Docker multi-stage build + docker-compose для local dev       |
| 1.0.0  | Янв. 2026 | Feature | Деплой на Hugging Face Spaces (порт 7860)                     |
| 1.0.0  | Янв. 2026 | Feature | Адаптивная вёрстка (мобильные от 320px)                       |
| 2.0    | Фев. 2026 | Docs    | Полная техническая документация v2.0 на основе исходного кода |

## 4. ЮРИДИЧЕСКАЯ ДОКУМЕНТАЦИЯ

### 4.1 Политика конфиденциальности

#### Бар «Пьяный Гейгер» — Политика обработки персональных данных

Настоящая политика описывает порядок обработки персональных данных пользователей сайта [q1est.github.io/mdk2/](https://q1est.github.io/mdk2/) (далее — «Сайт»). Оператор: бар «Пьяный Гейгер», Калининградская область, Форт № 1 «Штайн».

#### Какие данные собираются и хранятся

При использовании формы бронирования в PostgreSQL сохраняются следующие данные:

| Поле                       | Хранится в БД        | Цель сбора                                            |
|----------------------------|----------------------|-------------------------------------------------------|
| Имя/позывной (name)        | Да (TEXT)            | Идентификация гостя при бронировании                  |
| Телефон (phone)            | Да (TEXT)            | Связь для подтверждения брони                         |
| Дата визита (date)         | Да (DATE)            | Планирование посещения                                |
| Время визита (time)        | Да (TIME)            | Планирование посещения                                |
| Количество гостей (guests) | Да (INT)             | Подготовка столика нужного размера                    |
| E-mail                     | НЕТ — не сохраняется | Поле присутствует в форме, но не передаётся на сервер |
| Зона (zone)                | НЕТ — не сохраняется | Поле присутствует в форме, но не передаётся на сервер |
| Пожелания (comments)       | НЕТ — не сохраняется | Поле присутствует в форме, но не передаётся на сервер |
| created_at                 | Да (TIMESTAMP)       | Техническая метка времени создания записи             |

#### Права пользователя

- Запросить удаление своих персональных данных из БД.
- Получить информацию о составе хранимых данных.
- Отозвать согласие на обработку данных.
- Обратиться по телефону: +7 911 475-49-69.

Данные не передаются третьим лицам без согласия пользователя. Обработка осуществляется в соответствии с ФЗ №152 «О персональных данных». Дата вступления в силу: 1 февраля 2026 г.

## 4.2 Пользовательское соглашение

### Условия использования сайта бара «Пьяный Гейгер»

#### 1. Общие положения

Используя данный сайт, вы соглашаетесь с настоящими условиями. Сайт предоставляет информацию о баре и позволяет бронировать столики. Администрация вправе изменять содержимое без предварительного уведомления.

#### 2. Ограничение ответственности

Администрация не несёт ответственности за технические сбои и временную недоступность сайта. Данные корзины хранятся только в памяти браузера и не сохраняются между сессиями. Администрация не несёт ответственности за эффекты от употребления блюд.

#### 3. Бронирование

Отправка формы бронирования является заявкой, но не гарантирует наличие мест. Подтверждение осуществляется администратором по телефону +7 911 475-49-69.

#### 4. Интеллектуальная собственность

Тексты, изображения и дизайн сайта принадлежат бару «Пьяный Гейгер». Копирование без письменного разрешения запрещено.

#### 5. Применимое право

Соглашение регулируется законодательством РФ. Споры решаются по месту нахождения Оператора — Калининградская область, Форт № 1 «Штайн».

## 4.3 Акт приёма-передачи работ

### **АКТ ПРИЁМА-ПЕРЕДАЧИ РАБОТ по разработке веб-сайта бара «Пьяный Гейгер»**

г. Калининград

«\_\_» \_\_\_\_\_ 2026 г.

Исполнитель: \_\_\_\_\_ и Заказчик: Бар «Пьяный Гейгер» в лице  
\_\_\_\_\_ составили настоящий Акт о нижеследующем:

1. Перечень выполненных работ:

| № | Наименование работы                                                | Результат                   | Статус    |
|---|--------------------------------------------------------------------|-----------------------------|-----------|
| 1 | Разработка главной страницы (index.html)                           | Файл index.html             | Выполнено |
| 2 | Разработка страницы меню (menu.html) с корзиной                    | Файл menu.html              | Выполнено |
| 3 | Разработка таблицы стилей (styles.css)                             | Файл styles.css             | Выполнено |
| 4 | Разработка клиентских скриптов (script.js)                         | Файл script.js              | Выполнено |
| 5 | Разработка Go-бэкенда (main.go, handle.go, models.go, postgres.go) | 4 Go-файла                  | Выполнено |
| 6 | Настройка Docker (Dockerfile, docker-compose.yml)                  | 2 конфигурации              | Выполнено |
| 7 | Деплой Frontend на GitHub Pages                                    | URL: q1est.github.io/mdk2/  | Выполнено |
| 8 | Деплой Backend на Hugging Face Spaces                              | Docker-контейнер, порт 7860 | Выполнено |
| 9 | Подготовка технической документации v2.0                           | PDF-документ                | Выполнено |

2. Заказчик претензий по объёму, качеству и срокам выполнения работ не имеет.

3. Акт составлен в двух экземплярах, имеющих одинаковую юридическую силу.

| ИСПОЛНИТЕЛЬ    | ЗАКАЗЧИК       |
|----------------|----------------|
| ФИО: _____     | ФИО: _____     |
| Подпись: _____ | Подпись: _____ |
| Дата: _____    | Дата: _____    |
| М.П.           | М.П.           |